

Relazione di Testing

Catalogo Prodotti e Carrello — ActivityManagement · Oct 1, 2026 · @Claudio

1. Panoramica dell'architettura DB

La relazione copre le due funzionalità introdotte dalla Change Request sul lato cliente. Il Catalogo è la consultazione dei servizi e dei prodotti offerti dall'attività. Il Carrello è la selezione e la quantificazione degli item prima del checkout. Le due funzionalità poggiano su scelte architetturali diverse, che determinano la strategia di test.

1.1 Il Catalogo: la tabella servizio

Il Catalogo non ha una tabella dedicata: riusa la tabella `servizio` dello schema originale, estesa dalla Change Request. Un record rappresenta un servizio erogato in struttura oppure un prodotto spedibile; a distinguerli è la sola colonna `tipo`.

Colonna	Tipo	Origine	Ruolo nel Catalogo
<code>id</code>	INTEGER AUTO_INCREMENT	Schema base	Chiave primaria; identifica l'item nel carrello
<code>tipo</code>	ENUM('Servizio','Prodotto') NOT NULL	Change Request	Distingue il servizio in struttura dal prodotto spedibile
<code>nome</code>	VARCHAR(100) NOT NULL	Schema base	Etichetta mostrata a catalogo e nel carrello
<code>descrizione</code>	VARCHAR(1000) NOT NULL	Change Request	Descrizione estesa mostrata nella scheda item
<code>prezzo</code>	DOUBLE NOT NULL	Schema base	Prezzo unitario, base del calcolo del totale
<code>in_uso</code>	BOOLEAN	Schema base	Filtro di pubblicazione: solo <code>in_uso = 1</code> va a catalogo
<code>note</code>	VARCHAR(200)	Schema base	Testo libero mostrato nella scheda item

La definizione originale è in `CREATE TABLES.sql` (righe 49-57), l'estensione in `MODIFICHE DATABASE.sql` (righe 32-34). L'ENUM usa le maiuscole (`Servizio`, `Prodotto`), mentre le query del catalogo filtrano con `'servizio'` e `'prodotto'`. Il confronto funziona solo perché la collation predefinita di MySQL e MariaDB non distingue le maiuscole.

Nello schema non esiste alcuna colonna di giacenza o disponibilità: lo stock non è modellato né nel database né nell'applicazione. I casi di test sulla verifica delle scorte sono quindi non applicabili.

1.2 Il Carrello: stato applicativo, non tabella

Il Carrello non ha un corrispettivo nel database. È uno slice Redux definito in `CarrelloReducer.js`, registrato nello store con la chiave `carrello`, che vive solo nella memoria del browser. Lo stato iniziale è costruito con `getInitialStateWithoutStorage`, quindi non viene letto né scritto in `localStorage`. Lo slice contiene l'array `items`; ogni elemento copia `id`, `nome`, `prezzo`, `tipo`, `descrizione`, `note` e `quantita` dell'item scelto a catalogo.

Il carrello arriva nel database solo al checkout. `CheckoutView.jsx` serializza l'array in JSON e lo invia a `/INSERISCI_ORDINE` insieme al totale calcolato nel browser (righe 95 e 118-123). Il server lo scrive nella colonna `items` della tabella `ordine`.

Colonna di ordine	Tipo	Ruolo rispetto al Carrello
<code>codice</code>	<code>VARCHAR(26) NOT NULL</code>	Chiave primaria, generata dal server con <code>DATE_FORMAT(NOW(3), ...)</code>
<code>data_creazione</code>	<code>DATETIME NOT NULL</code>	Istante di creazione, usato dalle ricerche e dalla vista delle ultime 48 ore
<code>items</code>	<code>TEXT NOT NULL</code>	Contenuto del carrello serializzato in JSON
<code>metodo_pagamento</code>	<code>ENUM('Struttura','Spedizione','Corriere')</code>	Determina quali colonne successive sono valorizzate
<code>data_prenotazione,</code> <code>ora_prenotazione</code>	<code>DATE, TIME</code>	Valorizzate solo per il metodo Struttura
<code>indirizzo</code>	<code>VARCHAR(100)</code>	Valorizzato per Spedizione e Corriere
<code>numero_carta</code>	<code>VARCHAR(4)</code>	Ultime quattro cifre, solo per Spedizione
<code>totale</code>	<code>DOUBLE NOT NULL</code>	Somma di prezzo per quantità, calcolata nel browser
<code>is_pagato,</code> <code>data_conferma_pagamento</code>	<code>BOOLEAN DEFAULT 0, DATE</code>	Stato del pagamento; alimentano l'analisi economica
<code>id_cliente</code>	<code>INTEGER NOT NULL</code>	Chiave esterna verso <code>cliente</code>

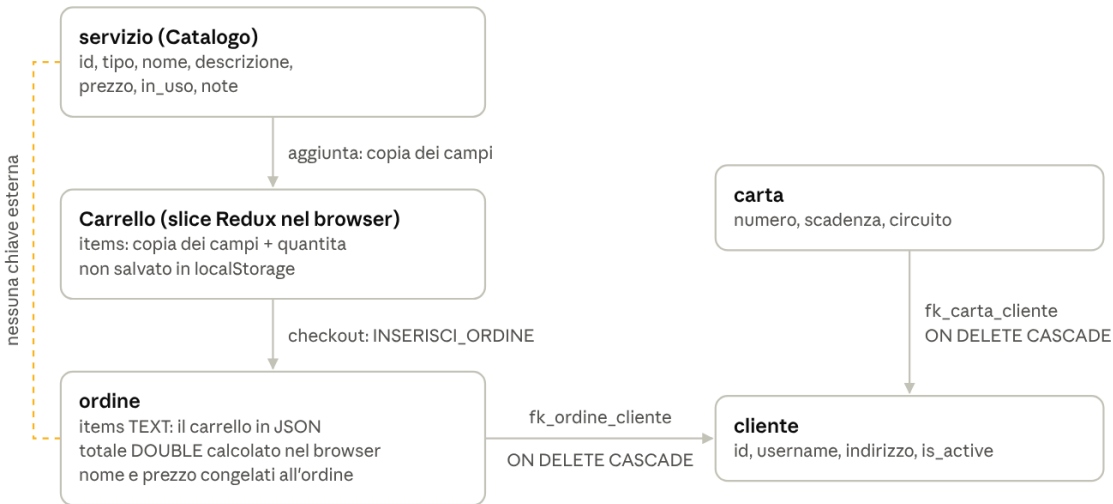
La tabella è definita in `MODIFICHE DATABASE.sql`, righe 36-57.

1.3 Relazioni e integrità referenziale

Gli ordini sono legati al cliente da una chiave esterna, ma al catalogo da nessun vincolo: il contenuto del carrello viene copiato nell'ordine come testo JSON.

Gli ordini dipendono dal cliente, non dal catalogo

Frecce piene: flusso dei dati e chiavi esterne con cascata · tratteggio: nessun vincolo



Relazioni fra catalogo, carrello, ordine e cliente

- `ordine.id_cliente` fa riferimento a `cliente.id` con **ON DELETE CASCADE** e **ON UPDATE CASCADE** (`fk_ordine_cliente`, MODIFICHE DATABASE.sql righe 52-56): eliminando un cliente si eliminano i suoi ordini.
- `carta.id_cliente` fa riferimento a `cliente.id` con lo stesso vincolo (`fk_carta_cliente`, righe 25-29).
- Fra gli item del carrello e `servizio.id` non esiste alcun vincolo. `ordine.items` è testo JSON denormalizzato, quindi il database non garantisce che un item ordinato esista ancora a catalogo, né propaga variazioni di prezzo o cancellazioni.
- La Change Request ha eliminato le tabelle `collegamento` e `lavoro` (righe 71-73). La relazione multi-a-molti che teneva le quantità in `collegamento.quantita` è stata sostituita dal JSON di `ordine.items`.

La scelta ha una conseguenza precisa. La denormalizzazione congela nome e prezzo al momento dell'ordine, il che è desiderabile per lo storico contabile, ma rinuncia a ogni integrità referenziale fra ordini e catalogo. Ogni verifica di coerenza fra i due deve quindi avvenire nell'applicazione, non nel database.

2. Strategia e piano di test

Il progetto dispone oggi di una suite Jest di 60 file e 585 test, che copre Actions, reducer, utilità e viste. La suite però non verifica le query del Catalogo, non avvia mai il server né il database, e non codifica

nessuno dei difetti noti. La strategia che segue affianca alla suite due banchi di prova mirati sulle due funzionalità: uno a livello unit e uno di integrazione.

2.1 Livelli di test, criteri di adeguatezza e strumenti

La verifica è organizzata su due livelli. Il livello unit isola la logica deterministica, che non dipende né dalla rete né dal database, e ne controlla il comportamento nominale e ai valori limite. Il livello di integrazione esercita la catena completa fino al database e ne verifica gli effetti osservabili. Il primo individua i difetti prima e a costo minore; il secondo stabilisce quali si manifestano davvero sul sistema in esecuzione.

I criteri di adeguatezza per considerare conclusa la verifica sono i seguenti:

- **Copertura funzionale del Carrello:** ogni reducer dello slice esercitato almeno una volta, con almeno un caso limite oltre a quello nominale.
- **Copertura delle query del Catalogo:** ogni query statica e ogni ramo del costruttore dinamico presenti in almeno un caso.
- **Copertura degli endpoint:** ogni endpoint HTTP dei due flussi invocato almeno una volta a server attivo. Sono sette: `VISUALIZZA_CATALOGO`, `VISUALIZZA_ITEMS`, `MODIFICA_ITEM`, `INSERISCI_ORDINE`, `OTTIENI_PAGAMENTI_DA_CONFERMARE`, `OTTIENI_NUMERO_PAGAMENTI_NON_CONFERMATI_CLIENTE` e `ELIMINAZIONE_PAGAMENTO_DA_CONFERMARE`.
- **Copertura dei vincoli:** ogni chiave esterna con propagazione in cascata verificata sperimentalmente (`fk_ordine_cliente` e `fk_carta_cliente`).
- **Robustezza:** ogni endpoint del Catalogo deve rispondere con un codice HTTP anche a una richiesta incompleta, senza interrompere il servizio.
- **Criterio di uscita:** nessun caso può restare con esito ignoto. Un caso non eseguibile va dichiarato tale, con la motivazione, e non conteggiato fra i superati.

Gli strumenti sono quelli già presenti fra le dipendenze del progetto o nell'ambiente di sviluppo, senza aggiunte.

Strumento	Versione	Impiego
Jest e babel-jest	30.5.2	Esecuzione della suite, del banco unit e misura della copertura
@babel/preset-env e preset-react	7.29.7	Trasformazione dei moduli ES e del codice JSX
jest-fixed-jsdom	0.0.11	Ambiente di test predefinito della configurazione
@testing-library/react e jest-dom	16.3.3 e 6.9.1	Test delle viste, già impiegati in <code>src/test-frontend</code>
msw	2.15.0	Simulazione delle risposte HTTP

Strumento	Versione	Impiego
		nei test delle viste
@stryker-mutator/core	9.6.1	Mutation testing sulle Actions; report del team in reports/mutation
Node con fetch nativo	24.21.0	Invocazione degli endpoint HTTP negli integration test
mysql2	3.24.5	Verifica diretta sul database e controllo dei vincoli
MariaDB (XAMPP)	10.4.32	Database degli integration test, in un'istanza isolata

Il documento di pianificazione del sistema originale indica anche Selenium e Cypress per il testing funzionale e di sistema. Nessuno dei due è fra le dipendenze del progetto, quindi i test end-to-end nel browser restano fuori dal perimetro. I test delle viste con Testing Library coprono in parte il livello funzionale, ma a livello di singolo componente e con le risposte del server simulate.

2.2 Unit test

Bersaglio: la logica che non dipende né dalla rete né dal database, cioè il reducer del Carrello e i costruttori delle query del Catalogo. Sono le parti deterministiche, quindi verificabili in isolamento.

- **Gestione degli item:** aggiunta a carrello vuoto; aggiunta di un item già presente, che deve incrementare la quantità senza duplicare la riga; aggiunta di item distinti; normalizzazione dei campi opzionali assenti.
- **Modifica delle quantità:** aggiornamento a un valore positivo e a zero, incremento e decremento unitari, rimozione automatica quando la quantità arriva a zero, rifiuto di una quantità nulla in aggiunta.
- **Rimozione e svuotamento:** rimozione per identificativo, rimozione di un identificativo inesistente, svuotamento dopo il checkout, persistenza tra un caricamento e l'altro della pagina.
- **Calcolo del totale:** somma di prezzo per quantità su più item, carrello vuoto, prezzi decimali, conteggio per il badge della navbar.
- **Query di catalogo:** filtro sugli item pubblicati, partizionamento per tipo, uso dei segnaposto, coerenza fra segnaposto e parametri, resistenza all'injection sul tipo, presenza della colonna descrizione.

Il totale e il badge non hanno una funzione dedicata: la formula è scritta dentro le viste. Il banco la riproduce identica a `CarrelloView.jsx` riga 30, `CheckoutView.jsx` riga 95 e `NavbarCliente.jsx` righe 27-31.

Il piano prevedeva anche la verifica della disponibilità a magazzino. Non è eseguibile perché la funzionalità non esiste: né la tabella servizio né le API espongono una giacenza, e il Carrello non limita la quantità selezionabile. Il caso resta a piano con esito Non applicabile.

2.3 Integration test

Bersaglio: la catena completa vista, Actions, endpoint HTTP, query e database. Si verifica che le query producano i risultati attesi su dati reali e che i vincoli dello schema si comportino come progettato.

- **Catalogo:** `VISUALIZZA_CATALOGO` restituisce i soli item con `in_uso = 1`; i filtri per prodotto e per servizio partizionano il catalogo; `VISUALIZZA_ITEMS` applica nome, fascia di prezzo, tipo e note; `MODIFICA_ITEM` aggiorna la descrizione.
- **Carrello e ordine:** `INSERISCI_ORDINE` serializza gli item e registra il totale; l'ordine nasce con `is_pagato = 0` e compare fra i pagamenti da confermare; una fascia oraria già piena viene rifiutata; il totale registrato è al centesimo.
- **Integrità:** la cancellazione di un cliente propaga in cascata su ordini e carte; la cancellazione di un item a catalogo non altera gli ordini già emessi.
- **Sicurezza e robustezza:** il parametro `tipo` non deve poter alterare la query; una richiesta incompleta non deve interrompere il server.

L'ambiente è stato costruito per non toccare dati reali. `storage/DB.js` contiene credenziali segnaposto e il database `activity_management` di XAMPP appartiene al team, quindi:

1. È stata avviata un'istanza separata di MariaDB 10.4.32, presa dall'installazione XAMPP, con una cartella dati nuova e in ascolto solo su `127.0.0.1:3306`.
2. Lo schema è stato creato eseguendo in ordine `CREATE DATABASE.sql`, `CREATE TABLES.sql` e `MODIFICHE DATABASE.sql`, senza errori.
3. I dati di prova sono un cliente e cinque item: due servizi e due prodotti pubblicati, più un prodotto ritirato (`in_uso = 0`).
4. L'API è una copia di `storage/` in una cartella temporanea, identica byte per byte all'originale tranne `DB.js`, che punta all'istanza isolata. Ascolta sulle porte 3000 e 8081 come quella del progetto.

Le scritture vengono annullate: gli ordini creati sono cancellati tramite l'API stessa, i controlli sui vincoli girano in transazioni chiuse con rollback. Un caso finale confronta i conteggi delle tabelle prima e dopo.

2.4 Approccio al testing di regressione

La Change Request non aggiunge soltanto funzionalità. Modifica la tabella `servizio`, elimina le tabelle `lavoro` e `collegamento` e riscrive parte dello strato di accesso ai dati condiviso con le funzionalità esistenti. Serve quindi un modo esplicito per accertare che ciò che non è stato toccato continui a funzionare.

L'approccio adottato è il retest all: la suite si riesegue per intero a ogni modifica. La scelta è motivata dai numeri: 60 suite e 585 test in circa 23 secondi. A questa scala una selezione automatica dei test costerebbe più del tempo risparmiato e rischierebbe di escludere un test rilevante. Una strategia selettiva converrebbe solo sopra qualche minuto di esecuzione.

Il retest all non esime dall'individuare l'area d'impatto, che dice dove guardare quando un test fallisce e quali suite mancano.

Modulo toccato dalla CR	Suite corrispondenti (casi)	Stato
-------------------------	-----------------------------	-------

Modulo toccato dalla CR	Suite corrispondenti (casi)	Stato
CarrelloReducer.js	new_version/.../CarrelloReducer.test.js (13)	Superata; non prova quantità zero né persistenza
CarrelloActions.js	new_version/.../CarrelloActions.test.js (9)	Superata
ServizioActions.js	new_version (11) e regression_testing (11)	Superate
ServizioReducer.js	new_version (11) e regression_testing (9)	Superate
ServizioSQL.js	regression_testing/.../ServizioSQL.test.js (2)	Superata, ma copre solo l'eliminazione: query del catalogo e costruttore di ricerca scoperti
OrdineActions.js, OrdineReducer.js	OrdineActions.test.js (21), OrdineReducer.test.js (1)	Superate
CartaActions.js, CartaReducer.js	CartaActions.test.js (9), CartaReducer.test.js (3)	Superate
CatalogoServiziView, CarrelloView, CheckoutView, ConfermaOrdine	test-frontend: Catalogo (2), Carrello (8), CheckoutView (12), ConfermaOrdine (2)	Superate; Actions e risposte del server simulate
store.js	regression_testing/.../store.test.js (5)	Superata
Controlli.js	new_version (62), regression_testing (12), test-frontend/controlli (23)	Superate
OrdineSQL.js, CartaSQL.js, AutenticazioneSQL.js, AttivitaSQL.js	Nessuna	Da creare
Endpoint di server.js	Nessuna suite automatica	Coperti solo dal banco di integrazione esterno (punto 3.3)
Moduli rimossi (Lavoro, Collegamento)	32 suite in src/test/old_version	Archivate ed escluse dalla configurazione

Il criterio di accettazione è questo: dopo ogni modifica la suite va rieseguita per intero, e l'insieme dei test superati non deve mai diminuire. Un test che passa da superato a fallito è una regressione e blocca l'avanzamento. Un test che fallisce per un percorso non più valido è un difetto della suite, e va corretto nella suite, non nel codice.

Oggi il criterio vale sull'intera parte client: tutte le 60 suite attive sono eseguibili e verdi. Non protegge però lo strato dati né il server, che la suite non esercita, e non segnala i difetti noti, perché nessun caso li codifica. Una suite verde non garantisce quindi che Catalogo e Carrello siano corretti.

3. Risultati ed esecuzione dei test

3.1 Esecuzione della suite Jest del progetto

La suite del repository è stata eseguita da terminale con `npm test`, dopo aver installato le dipendenze con `npm install`: la cartella `node_modules` era vuota. Tutte le suite e tutti i test sono superati.

```
Test Suites: 60 passed, 60 total
Tests:       585 passed, 585 total
Snapshots:   0 total
Time:        23.369 s
```

Una seconda esecuzione con misura della copertura ha dato lo stesso esito in 26,6 secondi. Nessun test è saltato o marcato `test.failing`.

Cartella	Suite	Test	Contenuto
src/test/new_version	21	276	Unit test di Actions, reducer e utilità, compresi Carrello, Ordine e Carta
src/test/regression_testing	20	157	Suite preesistenti riallineate alla cartella storage e al nuovo schema
src/test-frontend	19	152	Test delle viste con Testing Library e msw
Totale	60	585	

Le 32 suite numerate della versione originale (da 01_AutenticazioneForms a 34_Login) sono conservate in `src/test/old_version` ed escluse da `testPathIgnorePatterns` in `jest.config.mjs`. Eseguite a parte falliscono tutte in fase di import, senza raggiungere alcun test, perché puntano a percorsi che la ristrutturazione ha spostato o eliminato:

```
Cannot find module '../../../../../db/ServizioSQL.js' from '31_ServizioSQL.test.js'
Cannot find module '../../../../../store/slice/ServizioSlice.js' from
'16_ServizioSlice.test.js'
```

Sono un archivio storico e non incidono sull'esito di `npm test`. Il loro contenuto utile è confluito nelle cartelle `new_version` e `regression_testing`.

Delle 60 suite, 14 riguardano direttamente Catalogo, Carrello e ordine, per 113 test (elencate al punto 3.4). Nessuna di esse esercita le query di `SQL_SELEZIONE_CATALOGO` o di `sql_selezione_servizi`, né un endpoint reale: le viste e le Actions sono verificate con risposte del server simulate.

3.2 Verifica della logica di Catalogo e Carrello

I 24 casi del punto 2.2 sono stati eseguiti con Jest tramite un banco di prova costruito appositamente. Il banco importa e sollecita direttamente i moduli sorgente `CarrelloReducer.js` e `ServizioSQL.js`, senza modificarli. Ogni caso esprime il comportamento corretto atteso, quindi un fallimento indica un difetto presente nel codice. L'esito è di 24 casi eseguiti: 18 superati e 6 falliti.

Area	Casi	Superati	Falliti
Carrello: gestione item, quantità, rimozione, totale	17	14	3
Catalogo: query statiche e costruttore di ricerca	7	4	3
Totale	24	18	6

I sei fallimenti, con il valore osservato:

- **Quantità zero accettata:** `aggiungiAlCarrello` con `quantita: 0` crea una riga di carrello con quantità zero.
- **Totale decimale impreciso:** $0,10 + 0,20$ dà `0.30000000000000004`.
- **Carrello non persistente:** reimportando il modulo, come avviene a un ricaricamento della pagina, il carrello riparte vuoto.
- **Segnaposto e parametri disallineati:** `sql_selezione_servizi` genera 3 segnaposto, `params_selezione_servizi` restituisce 4 valori.
- **SQL injection sul tipo:** il valore `Servizio' OR '1'='1` compare tale e quale nella query generata.
- **Descrizione assente dal catalogo:** `SQL_SELEZIONE_CATALOGO` non seleziona la colonna `descrizione`.

Tre verifiche statiche aggiuntive riguardano rilievi che non hanno un caso unit dedicato.

`SQL_MODIFICA_SERVIZIO` non aggiorna `descrizione` (fallito). Nessun file client invoca `/VISUALIZZA_CATALOGO` (fallito). `ServizioActions.js` non richiama più l'azione inesistente `aggiornaCatalogo` (superato): il metodo `getCatalogo` che la usava è stato rimosso.

3.3 Esecuzione degli integration test a server attivo

Con MariaDB in ascolto sulla porta 3306 e l'API sulla porta 3000, i 20 casi del punto 2.3 sono stati eseguiti contro l'applicazione in funzione: 14 superati e 6 falliti. Le chiamate usano gli stessi campi che la vista invia al server, e ogni effetto è stato verificato direttamente sul database. I conteggi finali coincidono con quelli iniziali: un cliente, zero ordini e cinque item.

Area	Casi	Superati	Falliti
Catalogo	11	6	5

Area	Casi	Superati	Falliti
Carrello e ordine	6	5	1
Integrità referenziale	2	2	0
Controllo dell'ambiente	1	1	0
Totale	20	14	6

SQL injection confermata. Interrogando `VISUALIZZA_ITEMS` come cliente autenticato con il tipo legittimo `Servizio`, la risposta contiene 2 righe, entrambe servizi. Con il valore manipolato ne contiene 4, servizi e prodotti insieme:

```
tipo = "Servizio"          -> 2 righe, tutte di tipo Servizio (corretto)
tipo = "Servizio' OR '1'='1" -> 4 righe, di tipo Servizio e Prodotto (clausola WHERE alterata)
```

La condizione iniettata, per la precedenza di `AND` su `OR`, restituisce ogni item pubblicato e annulla il filtro scelto dall'utente. Dalla vista il valore non arriva: `controlloRicercaServizi` accetta per il tipo solo lettere (`/^[a-zA-Z]+$`). L'endpoint però è privo di autenticazione e di validazione lato server, quindi basta una richiesta HTTP diretta.

Una sola richiesta abbatte l'API. Una ricerca inviata senza i campi `in_uso` e `id_cliente` fa sollevare a `ServizioSQL.js` riga 110 un'eccezione non gestita. Il processo Node termina e il server smette di rispondere a tutti gli utenti finché qualcuno non lo riavvia:

```
TypeError: Cannot read properties of undefined (reading 'toLowerCase')
    at ServizioSQL.sql_selezione_servizi (storage/ServizioSQL.js:110:47)
    at storage/server.js:265:25
Node.js v24.21.0 (processo terminato con codice 1)
```

L'eccezione nasce prima del blocco `try` dell'endpoint. Express 4 non intercetta gli errori dei gestori `async`, quindi il rifiuto della promessa resta non gestito e Node chiude il processo. Il difetto non era emerso dai test unit, che passano sempre un oggetto di ricerca completo.

Gli altri quattro fallimenti confermano a runtime difetti già visti nel codice:

- **Filtro sulle note ignorato:** con `note = "secchi"` l'endpoint risponde 200 e restituisce 4 righe invece di 1. Il quarto parametro viene scartato senza errori.
- **Descrizione non modificabile:** `MODIFICA_ITEM` risponde 200, ma la colonna `descrizione` resta quella originale.
- **Descrizione assente dal catalogo:** la risposta di `VISUALIZZA_CATALOGO` contiene solo `id`, `nome`, `prezzo`, `tipo` e `note`.
- **Totale non arrotondato:** un ordine da 0,10 + 0,20 viene registrato nella colonna `totale` come 0.30000000000000004.

I 14 casi superati coprono tutto il flusso dell'ordine. L'ordine viene serializzato e registrato con totale 35,00, nasce con `is_pagato = 0` e compare fra i pagamenti da confermare, con il contatore del cliente a 1. Viene poi cancellato tramite API. Una seconda prenotazione sulla stessa fascia oraria piena viene rifiutata con `problema = true`. Anche i vincoli funzionano come progettato: eliminando il cliente

spariscono in cascata i suoi ordini e le sue carte, mentre eliminando un item a catalogo gli ordini già emessi conservano nome e prezzo storici.

Un'osservazione esterna al perimetro: `SQL_OTTIENI_ENTRATE_ORDINI`, che alimenta l'analisi economica a partire dagli ordini, usa `JSON_TABLE`. MariaDB 10.4, la versione inclusa in XAMPP, non supporta questa funzione e restituisce un errore di sintassi. L'analisi delle entrate funziona quindi solo su MySQL 8 o su MariaDB 10.6 e successive.

3.4 Casi della suite di progetto su Catalogo e Carrello

Le 14 suite del progetto che toccano le due funzionalità contano 113 test, tutti superati. Verificano il comportamento nominale con cura; nessuna verifica i casi in cui il banco di prova ha trovato difetti.

Suite	Test	Cosa verifica	Cosa non verifica
<code>new_version/.../CarrelloReducer.test.js</code>	13	I sei reducer, con item presente e assente	Quantità zero in aggiunta, persistenza
<code>new_version/.../CarrelloActions.test.js</code>	9	Guardia quantità ≤ 0 nelle Actions, aggiunta multipla, dispatch delle altre azioni	–
<code>test-frontend/Carrello.test.js</code>	8	Visualizzazione, incremento, decremento, rimozione, totale con prezzi interi, navigazione al checkout	Totale con prezzi decimali
<code>test-frontend/Catalogo.test.js</code>	2	Ricerca con risultati e ricerca vuota	La query: <code>ServizioActions</code> è sostituita da un mock
<code>test-frontend/CheckoutView.test.js</code>	12	Metodi di pagamento, carte, campi obbligatori, fasce orarie sature, errore dell'API, payload dell'ordine	Il server: risposte simulate con msw
<code>test-frontend/ConfermaOrdine.test.js</code> e <code>ordini-actions.test.js</code>	3	Pagina di conferma, ritorno allo shop, date della ricerca ordini	–

Suite	Test	Cosa verifica	Cosa non verifica
new_version/.../OrdineActions.test.js e OrdineReducer.test.js	22	Inserimento con fascia piena e libera, pagamenti da confermare, conferma, eliminazione, ricerca, ordini delle ultime 48 ore	Il server: fetch simulata
ServizioActions.test.js (new_version e regression_testing)	22	Inserimento, ricerca, modifica, selezione ed eliminazione dei servizi	Il contenuto della query eseguita
ServizioReducer.test.js (new_version e regression_testing)	20	Aggiornamento della lista, inserimento, modifica, tipo di selezione, stato prima e dopo la modifica	–
regression_testing/.../ServizioSQL.test.js	2	Query e parametri di eliminazione	Tutte le query del catalogo e della ricerca

Mutation testing. Il repository contiene tre report di Stryker prodotti dal team (reports/mutation), relativi alle nove classi Actions. Non sono stati rieseguiti: la configurazione ha coverageAnalysis: "off" e concorrenza 1, quindi ogni mutante rilancia l'intera suite: con 746 mutanti e circa 25 secondi per esecuzione servirebbero circa cinque ore.

Classe Actions	Mutanti (esecuzione 3)	Punteggio esecuzione 1	Punteggio esecuzione 3
CarrelloActions.js	26	92,31 % (2 sopravvissuti)	100 %
ServizioActions.js	196	79,08 % (41 sopravvissuti)	100 %
OrdineActions.js	59	67,24 % (19 sopravvissuti)	100 %
Tutte e nove le classi	746	80,24 % (147 sopravvissuti)	100 %

Fra la prima e la terza esecuzione il punteggio complessivo sale dall'80,24 % al 100 %: ogni mutante sopravvissuto è stato poi ucciso da un test. Il dato va però letto insieme al perimetro di Stryker, che muta soltanto le Actions. Dei nove rilievi del punto 4.2 nessuno risiede nelle Actions: sono in CarrelloReducer.js, ServizioSQL.js, server.js e nelle viste. Un punteggio pieno sulle Actions non dice quindi nulla sulla correttezza di questi moduli.

3.5 Tabella di riepilogo

Il riepilogo raccoglie i 44 casi eseguiti, 24 unit e 20 di integrazione: 32 superati e 12 falliti. In coda si aggiungono il caso non applicabile sulla disponibilità a magazzino e due righe di contesto sulla suite del progetto, non conteggiate nei 44.

Modulo	Tipo di test	Caso di test	Esito
Carrello	Unit	Aggiunta di un item a carrello vuoto	Passato
Carrello	Unit	Aggiunta di un item già presente: incrementa senza duplicare la riga	Passato
Carrello	Unit	Aggiunta di item distinti: due righe separate	Passato
Carrello	Unit	Campi opzionali assenti normalizzati a stringa vuota	Passato
Carrello	Unit	Il reducer rifiuta una quantità pari a zero	Fallito
Carrello	Unit	Rimozione di un item per identificativo	Passato
Carrello	Unit	Rimozione di un identificativo inesistente: stato invariato	Passato
Carrello	Unit	Aggiornamento della quantità a un valore positivo	Passato
Carrello	Unit	Aggiornamento della quantità a zero: l'item viene rimosso	Passato
Carrello	Unit	Incremento della quantità di una unità	Passato
Carrello	Unit	Decremento fino a zero: l'item viene rimosso	Passato
Carrello	Unit	Svuotamento del	Passato

Modulo	Tipo di test	Caso di test	Esito
		carrello dopo il checkout	
Carrello	Unit	Calcolo del totale su più item ($12,50 \times 2 + 10,00 \times 3 = 55,00$)	Passato
Carrello	Unit	Totale di un carrello vuoto pari a zero	Passato
Carrello	Unit	Totale con prezzi decimali privo di errori di arrotondamento	Fallito
Carrello	Unit	Conteggio degli item per il badge della navbar	Passato
Carrello	Unit	Persistenza del carrello tra i ricaricamenti di pagina	Fallito
Catalogo	Unit	Il catalogo espone i soli item con <code>in_uso</code> pari a 1	Passato
Catalogo	Unit	Filtro del catalogo sui soli prodotti	Passato
Catalogo	Unit	Filtro del catalogo sui soli servizi	Passato
Catalogo	Unit	La ricerca usa segnaposto per nome e fascia di prezzo	Passato
Catalogo	Unit	Numero di segnaposto coerente con i parametri passati	Fallito
Catalogo	Unit	Il filtro sul tipo non permette SQL injection	Fallito
Catalogo	Unit	Il catalogo espone la descrizione mostrata dall'interfaccia	Fallito

Modulo	Tipo di test	Caso di test	Esito
Catalogo	Integrazione	VISUALIZZA_CATALOGO restituisce i soli item pubblicati	Passato
Catalogo	Integrazione	I filtri prodotto e servizio partizionano il catalogo	Passato
Catalogo	Integrazione	Il catalogo restituisce la descrizione mostrata dall'interfaccia	Fallito
Catalogo	Integrazione	VISUALIZZA_ITEMS risponde alla ricerca del cliente con i filtri vuoti	Passato
Catalogo	Integrazione	VISUALIZZA_ITEMS filtra correttamente per nome	Passato
Catalogo	Integrazione	VISUALIZZA_ITEMS filtra per fascia di prezzo	Passato
Catalogo	Integrazione	VISUALIZZA_ITEMS filtra per tipo con un valore legittimo	Passato
Catalogo	Integrazione	Il filtro sul tipo non permette SQL injection (endpoint reale)	Fallito
Catalogo	Integrazione	Il filtro sulle note viene applicato dalla ricerca	Fallito
Catalogo	Integrazione	La descrizione di un item si aggiorna con MODIFICA_ITEM	Fallito
Catalogo	Integrazione	Una ricerca priva di in_uso e id_cliente non abbatte il server	Fallito
Carrello	Integrazione	INSERISCI_ORDINE serializza gli item e registra il totale	Passato

Modulo	Tipo di test	Caso di test	Esito
Carrello	Integrazione	L'ordine creato risulta con pagamento da confermare	Passato
Carrello	Integrazione	L'ordine compare fra i pagamenti da confermare	Passato
Carrello	Integrazione	Eliminazione dell'ordine di prova tramite API	Passato
Carrello	Integrazione	Una prenotazione su una fascia oraria già piena viene rifiutata	Passato
Carrello	Integrazione	Il totale registrato è arrotondato al centesimo	Fallito
Carrello	Integrazione	La cancellazione di un cliente propaga in cascata su ordini e carte	Passato
Catalogo	Integrazione	La cancellazione di un item a catalogo non altera gli ordini emessi	Passato
Ambiente	Integrazione	I conteggi delle tabelle sono invariati al termine	Passato
Catalogo	Unit	Verifica della disponibilità a magazzino	Non applicabile
Suite Jest	Regression	Esecuzione delle 60 suite del progetto (585 test)	Passato
Suite Jest	Regression	Copertura dei difetti noti di Catalogo e Carrello da parte della suite	Assente

4. Note e valutazioni finali

4.1 Stabilità delle funzionalità testate

Il Carrello è solido nel comportamento nominale: 14 casi unit su 17 superati, e 5 casi di integrazione su 6. Aggiunta, incremento, decremento, rimozione, svuotamento e calcolo del totale si comportano come previsto dai casi d'uso, compresi i casi limite. Il flusso dell'ordine funziona dall'inizio alla fine: serializzazione, registrazione, pagamento da confermare, rifiuto della fascia piena e cancellazione. I tre difetti riguardano robustezza e precisione, e nessuno impedisce di completare un ordine.

Il Catalogo è la parte più fragile: 4 casi unit superati su 7 e 6 di integrazione su 11. Le query statiche sono corrette e parametriche. Il costruttore della ricerca dinamica concentra invece i due difetti più gravi dell'analisi: l'injection sul tipo e il crash del server su una richiesta incompleta.

I vincoli di integrità si comportano come progettato. La cancellazione di un cliente elimina in cascata ordini e carte. La cancellazione di un item lascia intatti gli ordini già emessi, confermando l'effetto della denormalizzazione descritta al punto 1.3.

4.2 Dettaglio dei rilievi emersi

I rilievi sono nove: uno critico, uno alto, cinque medi e due bassi. Sei sono stati osservati sul sistema in esecuzione, gli altri tre ricavati dal codice e dai test unit.

Rilievo	Gravità	Descrizione e riferimento	Accertato
SQL injection sul filtro tipo	Critica	ServizioSQL.js riga 107 concatena <code>params.tipo</code> nella query. Con Servizio' OR '1'='1 l'endpoint VISUALIZZA_ITEMS restituisce 4 righe di entrambi i tipi invece di 2 servizi. L'endpoint non richiede autenticazione e non valida l'input lato server.	In esecuzione
Crash del server su richiesta incompleta	Alta	ServizioSQL.js riga 110 chiama <code>toLowerCase()</code> su <code>params.in_uso</code> senza verificarne la presenza. Una richiesta senza <code>in_uso</code> e <code>id_cliente</code> solleva un'eccezione	In esecuzione

Rilievo	Gravità	Descrizione e riferimento	Accertato
		fuori dal try (server.js riga 265) e il processo Node termina. Il servizio resta fermo per tutti gli utenti fino al riavvio.	
Il catalogo sicuro non viene usato	Media	Nessun file client invoca /VISUALIZZA_CATALOGO, l'unico endpoint del catalogo interamente parametrico. CatalogoServiziView.jsx riga 70 passa da ricercaServizi e quindi dal percorso vulnerabile. Il metodo getCatalogo, che chiamava un'azione inesistente del reducer, è stato rimosso.	Nel codice
Descrizione non modificabile	Media	SQL_MODIFICA_SERVIZIO (righe 64-71) non aggiorna la colonna descrizione: MODIFICA_ITEM risponde 200 ma la descrizione resta quella iniziale.	In esecuzione
Segnaposto e parametri disallineati	Media	sql_selezione_servizi genera 3 segnaposto, params_selezione_servizi ne fornisce 4. L'endpoint risponde 200, scarta il parametro in eccesso e non applica mai il filtro sulle note.	In esecuzione

Rilievo	Gravità	Descrizione e riferimento	Accertato
Descrizione assente dal catalogo	Media	SQL_SELEZIONE_CATALOGO non seleziona descrizione, che CatalogoServiziView.jsx riga 97 mostra. Oggi non si vede perché la vista usa la ricerca, ma emerge appena la si collega a /VISUALIZZA_CATALOGO.	In esecuzione
Carrello non persistente	Media	CarrelloReducer.js usa getInitialStateWithoutStorage e non chiama saveToLocalStorage, a differenza di AutenticazioneReducer: ricaricando la pagina il cliente perde la selezione.	Test unit
Nessuna guardia sulla quantità nel reducer	Bassa	Il controllo sulla quantità non positiva vive solo in CarrelloActions.js riga 24. Il reducer accetta zero e crea una riga di carrello senza significato.	Test unit
Imprecisione del totale in virgola mobile	Bassa	0,10 + 0,20 dà 0.30000000000000004, e questo valore viene scritto nella colonna totale. In interfaccia toFixed(2) lo maschera. La formula è duplicata in CarrelloView.jsx riga 30 e CheckoutView.jsx	In esecuzione

Rilievo	Gravità	Descrizione e riferimento	Accertato
		riga 95.	

Il rilievo sul catalogo sicuro era classificato Alto nella relazione dell'08/08/2026, perché `getCatalogo` avrebbe sollevato un errore. Rimosso quel metodo, resta un problema di progettazione che non provoca guasti da solo: per questo è ora classificato Medio. Il crash del server, invece, è emerso per la prima volta in questa verifica.

4.3 Copertura del codice

La suite del progetto copre l'86,17 % delle istruzioni di `src/react_redux`, il 79,64 % dei rami e l'85,94 % delle righe. Il dato è misurato da Jest con `npm test -- --coverage`. Cinque dei sette moduli di Catalogo e Carrello raggiungono il 100 % delle istruzioni.

Modulo sorgente	Istruzioni	Istruzioni coperte	Rami coperti
<code>CarrelloReducer.js</code>	36	100 %	87,5 %
<code>CarrelloActions.js</code>	14	100 %	100 %
<code>CarrelloView.jsx</code>	24	100 %	100 %
<code>CheckoutView.jsx</code>	140	85,0 %	71,2 %
<code>CatalogoServiziView.jsx</code>	36	77,8 %	50 %
<code>ServizioActions.js</code>	106	100 %	100 %
<code>ServizioReducer.js</code>	45	100 %	100 %

La tabella seguente indica, componente per componente, cosa risulta coperto e cosa no.

Componente	Copertura	Nota
<code>CarrelloReducer.js</code>	Completa	Tutti e sei i reducer sollecitati; restano scoperti i valori predefiniti di descrizione e note assenti (righe 26-27), che solo il banco verifica
Calcolo del totale	Verificato	La formula è duplicata in <code>CarrelloView.jsx</code> riga 30 e <code>CheckoutView.jsx</code> riga 95: non esiste un punto unico da testare
<code>ServizioSQL.js</code> (catalogo)	Completa nel banco, assente nella suite	Le tre query statiche e il costruttore dinamico sono verificati dal banco; la suite

Componente	Copertura	Nota
		copre solo l'eliminazione. Non quantificata perché fuori da <code>collectCoverageFrom</code>
<code>CarrelloActions.js</code> e <code>ServizioActions.js</code>	Completa	100 % delle istruzioni e punteggio di mutazione del 100 % nell'ultimo report
<code>CatalogoServiziView.jsx</code>	Parziale	Metà dei rami scoperti; <code>ServizioActions</code> è simulata
Endpoint di catalogo e ordine	Coperti dal banco	7 endpoint su 7 invocati a server attivo; nessuna suite automatica nel repository

La configurazione di Jest raccoglie la copertura solo da `src/react_redux.storage/`, dove risiedono le query e il server, e `src/utils/` restano esclusi: la copertura dello strato dati non è misurata. Per renderla visibile bisogna estendere `collectCoverageFrom`.

Una copertura del 100 % non implica l'assenza di difetti. `CarrelloReducer.js` ha tutte le istruzioni coperte ma accetta ancora la quantità zero, perché nessun caso della suite ne verifica il rifiuto.

4.4 Fatti rilevanti e deviazioni dal piano

L'attività si è discostata dal piano su cinque punti, tutti documentati nelle sezioni precedenti.

Deviazione	Causa	Effetto sul piano
Verifica della disponibilità a magazzino non eseguita	La funzionalità non è modellata né nel database né nell'applicazione	Caso dichiarato non applicabile anziché superato
Integration test su un'istanza isolata di MariaDB	<code>storage/DB.js</code> contiene credenziali segnaposto; il database del team non andava modificato	Dati di prova controllati e risultati riproducibili; non verificati i dati reali del team
API eseguita da una copia di <code>storage/</code>	Serviva puntare l'API all'istanza isolata senza modificare il progetto	Copia verificata identica byte per byte, salvo <code>DB.js</code>
Casi eseguiti con banchi di prova esterni al repository	La suite non contiene casi per le query del Catalogo, per il server e per i difetti noti	Nessun file del progetto modificato; i casi non proteggono ancora da regressioni
Mutation testing non rieseguito	Con <code>coverageAnalysis</code> disattivato servirebbero circa	Riportati i report già presenti nel repository

Deviazione	Causa	Effetto sul piano
	cinque ore	

Sul piano quantitativo sono stati eseguiti 44 casi a fronte dei 45 pianificati, con un caso non applicabile: 32 superati e 12 falliti. La suite del progetto conta 60 file e 585 test, tutti superati in circa 23 secondi. Sono stati individuati nove difetti distinti, di cui uno critico e uno alto; sei sono stati osservati sul sistema in esecuzione.

Il rilievo di processo più significativo è la capitalizzazione dei casi. La suite è interamente verde, ma nessuno dei suoi 585 test fallisce in presenza dei nove difetti, e nessuno avvia il server o il database. Finché i casi di questa relazione restano fuori dal repository, una correzione o una regressione su Catalogo e Carrello passerebbe inosservata.

4.5 Conclusioni e raccomandazioni

La logica di stato del Carrello è funzionalmente corretta e può considerarsi stabile, e l'esecuzione a server attivo lo conferma sull'intero flusso dell'ordine. I suoi difetti riguardano la robustezza e la precisione, non il comportamento nominale.

Il Catalogo richiede invece un intervento prima di qualsiasi esposizione pubblica dell'applicazione. Il costruttore della ricerca ha due difetti dimostrati sul sistema in funzione: un'injection che altera la query e un input incompleto che spegne il server. Non vanno trattati come rischi potenziali.

Gli interventi consigliati, in ordine di priorità:

1. Sostituire la concatenazione di `ServizioSQL.js` riga 107 con un segnaposto, validando tipo lato server contro i soli valori `Servizio` e `Prodotto`.
2. Rendere `sql_selezione_servizi` tollerante ai campi mancanti (`in_uso` assente) e intercettare gli errori dei gestori `async` di `Express`, con un `try` che includa la costruzione della query o un middleware di gestione degli errori.
3. Collegare `CatalogoServiziView` a `/VISUALIZZA_CATALOGO` e, nello stesso intervento, aggiungere `descrizione` a `SQL_SELEZIONE_CATALOGO`: le due modifiche vanno fatte insieme, altrimenti la descrizione sparisce dalla vista. Serve una nuova azione e il relativo reducer, dato che `getCatalogo` è stato rimosso.
4. Estendere `SQL_MODIFICA_SERVIZIO` e `params_modifica_servizio` alla colonna `descrizione`.
5. Allineare `params_selezione_servizi` ai segnaposto della query, aggiungendo il filtro `note LIKE ?` oppure rimuovendo il quarto parametro.
6. Persistere il carrello su `localStorage` con lo schema già usato da `AutenticazioneReducer`, e spostare nel reducer la guardia sulla quantità non positiva.
7. Estrarre il calcolo del totale in una funzione unica, arrotondata a due decimali prima dell'invio a `/INSERISCI_ORDINE`.
8. Portare nella suite del progetto i 44 casi di questa relazione: i casi unit con `test.failing` finché il difetto è presente, gli integration test in una configurazione dedicata (`npm run test:integration`) con un database di test.
9. Estendere `collectCoverageFrom` a `storage/` e `src/utills/`, così da misurare anche la copertura dello strato dati.

10. Fuori perimetro: allineare la versione del database di sviluppo, perché l'analisi delle entrate usa `JSON_TABLE`, assente in MariaDB 10.4.